

面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v26: RevKit 高维超时边界版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合把经典谓词嵌入量子算法, 是 Grover 搜索、量子密码分析和约束求解类量子程序中的基础编译步骤。现有路线通常围绕固定布尔表示、可逆逻辑模板或 CNOT 导向覆盖结构展开, 但容错量子计算中的逻辑开销同时受 T-count、CNOT、深度、门数和辅助比特制约。本文将该任务重新表述为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed–Muller (fixed-polarity Reed–Muller, FPRM) 项集合上的资源约束搜索问题, 提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 的 Resource-NMCTS 框架。本文方法不依赖 SSHR 的 parallelotope 候选空间; SSHR 仅作为 small-scale CNOT-oriented baseline 参与比较。

实验显示, 在 $n \leq 6$ 的 177 个传统函数上, Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%; 相对 weighted ESOP-MILP 获得 167/3/7, 平均 score 降低 29.84%。新增 ROS-style LUT proxy 对 $n = 3-6$ 与 $n = 14, 15, 16, 18$ 共 309 个函数运行 ABC $K = 3, 4, 5$ LUT sweep, 927/927 行均通过 truth-table 检查; best- K proxy 相对固定 $K = 4$ 平均 score 降低 18.12%, 而 Resource-NMCTS 相对该更强 proxy 仍获得 309/0/0, 平均 score 降低 83.77%。进一步安装并运行 RevKit Python API 的 `oracle_synth`, 在 $n \leq 6$ 的 177 个传统函数上全部合成成功; 但新的角度审计显示 RevKit 返回的是含 R_z 旋转的 phase netlist, 而不是可直接按 T/Tdg 完整计数的 Clifford+T netlist: 177 行中只有 6 行不含非 Clifford R_z , 171 行含非 Clifford R_z , 总数为 9242, 角度除以 π 的最大分母为 64。因此, Resource-NMCTS 相对 RevKit 的 6/171/0、平均 score 高 751.69% 和 T-like count 高 4060.08% 只能解释为 RevKit Rz-phase lower-bound 口径下的压力结果, 不能解释为精确 Clifford+T T-count 结果。若对每个非 Clifford R_z 仅加入 1 个 score 单位的符号成本, Resource-NMCTS 相对 RevKit 转为 140/37/0、平均 score 降低 14.52%, Pareto-Resource-NMCTS 转为 157/20/0、平均 score 降低 17.89%; 符号成本为 2/Rz 时二者均达到 177/0/0。进一步把 Resource-NMCTS、Pareto-Resource-NMCTS 和 FPRM polarity archive 作为 Resource-NMCTS family score-reranked portfolio 后, $\lambda_{R_z} = 1$ 时为 157/20/0, $\lambda_{R_z} = 1.5$ 时已达到 177/0/0, 而传统 baseline family 在 $\lambda_{R_z} = 1$ 时只有 80/97/0。若采用 Ross–Selinger 风格的 R_z 近似综合 proxy, 按 $T/R_z = \lceil 3 \log_2(1/\epsilon) \rceil$ 计费, 则在 $\epsilon = 10^{-3}$ 时 $T/R_z = 30$, Resource-NMCTS family 相对 RevKit synthesized-cost proxy 为 177/0/0, 平均 score 降低 95.03%; 更保守的 $4 \log_2(1/\epsilon) + 10$ proxy 在 $\epsilon = 10^{-6}$ 时同样为 177/0/0, 平均 score 降低 97.01%。这些仍是逻辑层成本模型, 不是实际 rotation sequence 输出。进一步的 $n = 14$ RevKit 高维隔离超时探针显示, 8 个函数中 1 个在 30 s 内返回、7 个超时; 唯一返回行含 32767 个非 Clifford R_z , 在 RevKit lower-bound score 下 Resource-NMCTS 相对较差, 但加入 1/Rz 符号成本后 Resource-NMCTS 转为胜出。这说明当前正式 RevKit paired benchmark 应限制在已验证的 $n \leq 6$, 高维 RevKit API 只能作为可扩展性边界证据。在 $n = 18$ 函数级切片上, 完整 Resource-NMCTS 相对 adaptive depth-2 Boolean-ring screen 进一步降低 23.85% 平均 score; 在 $n = 20$ 边界切片上, adaptive depth-2

Boolean-ring screen 相对 AND-direct ANF 降低 11.80% 平均 score。

项集级 $n = 20, 22, 24, 28, 32, 36, 40$ scale 实验表明, 结构深度策略可以在接近 all-depth adaptive 质量的同时减少约三分之一时间; depth-frontier policy 在 $n = 20, 28, 40$ 上相对 fixed depth-2 获得 35/0/37, 平均 score 降低 2.19%, 相对 all-depth depth ≤ 4 节省 58.69% 时间。独立 seed 的 $n = 24, 28, 32, 40$ 泛化实验中, 原 frontier policy 相对 fixed depth-2 为 40/0/56、平均 score 降低 1.85%; 扩大训练集后的 large frontier policy 在同一泛化集上提升到 56/0/40、平均 score 降低 2.34%, 相对旧 policy 为 17/0/79、平均 score 再降 0.49%, 相对 all-depth 仅高 0.10% 且节省 53.50% 时间。进一步加入时间感知标签目标 $score_{\Delta} + 0.003 time_{\Delta}$ 后, cost-aware frontier policy 在同一泛化集上仍相对 fixed depth-2 获得 56/0/40、平均 score 降低 1.39%, 但把相对 fixed depth-2 的 plan-time 增幅从 large policy 的 563.80% 降至 170.03%; 相对 large policy, 它以 +0.99% score 代价换取 33.02% 时间下降和 7.61% 辅助线 lifetime area 下降。

所有 6600 个项集级方法行均通过 factor-plan ANF 展开验证和 emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证; 进一步的 $n = 21, 22, 23$ bridge、large-policy $n = 23$ rerun 和 cost-aware $n = 23$ rerun 中, 300/300 个方法行同时通过完整 truth-table oracle 验证、plan 验证和 emitted-circuit 符号验证。新增 emitted-circuit schedule proxy 显示, large frontier policy 在 $n = 23$ 完整 bridge 上相对旧 policy 为 1/0/5, 平均 score 再降 0.48%, T-depth proxy 再降 0.45%; cost-aware policy 则相对 large policy 以 +0.92% score 代价换取 56.29% plan time 下降和 12.62% lifetime area 下降。本文的阶段性结论是: Resource-NMCTS 已形成一条独立于 SSHR、以低 T-count 和低加权逻辑资源为目标的 Boolean oracle synthesis 主线; 但 RevKit API 结果表明, 论文若要投稿为强算法贡献, 还需要增加 phase/Rz-aware 口径并处理非 Clifford 旋转综合成本, 或者明确限制 bit-flip oracle 目标, 并继续补官方 ROS/mockturtle、legacy RevKit/CirKit 和后端相关评估。

关键词: 量子 oracle 综合; 布尔函数综合; 强化学习; 蒙特卡洛树搜索; ANF; FPRM; 布尔环; 资源约束编译

作者想法整理与当前论文定位

本稿首先服务于一个清晰的研究主题: 基于强化学习与蒙特卡洛树搜索的量子布尔函数综合方法。这个主题的本质不是复现某个已有综合算法, 而是把 Boolean oracle synthesis 看作搜索问题: 给定布尔函数的 ANF 或 FPRM 项集合, 系统需要在大量可计算、可反算、可验证的代数分解动作中选择一条低资源线路生成路径。强化学习、神经排序和 MCTS 的作用, 是在候选动作过多时更早找到有资源收益的分解, 而不是直接生成不可解释的量子门序列。

因此, 本文不应从 SSHR 入手。SSHR 的 paralleloptope 结构适合做 small-scale CNOT-oriented synthesis, 适合作为重要 baseline, 但它不是本文方法的依赖, 也不应成为论文主线。更合适的主线是: ANF/FPRM 项集合搜索定义问题, Boolean-ring screen 扩展可用结构, 神经动作先验和 MCTS 处理动作组合, depth policy 与 frontier policy 处理搜索预算, guard 保证学习策略不会轻易劣于确定性 baseline。这样的叙事可以自然比较 T-count、CNOT、门数、线路深度和辅助比特, 而不是被单个 CNOT 指标锁死。

从投稿角度看, 本文当前已有“明显提升”的证据, 但证据形态必须如实呈现。小规模传统函数上, 本文在低 T-count 和加权 score 上相对 ESOP、weighted ESOP-MILP、direct ANF 和 AND-direct ANF 有稳定优势; 高维部分已经从单纯资源估计推进到项集级 plan 验证、emitted-circuit GF(2) 符号验证和 $n = 21, 22, 23$ 完整 truth-table bridge。与此同时, 本文还不能声称硬件后端最

优，也不能声称 CNOT-only 全面超过 SSHR。当前稿件应把优势写成“资源约束逻辑层搜索”，把边界写成“仍需外部工具链和后端评估”。

下一步最值得投入的方向也很明确。第一，RevKit API baseline 已经显示 Rz-phase netlist lower-bound 口径很强，但其中 171/177 行含非 Clifford R_z ，且新的 $n = 14$ timeout probe 显示高维 API 比较必须用隔离超时而不能人工中止；因此需要新增 phase/Rz-aware 的内部 emitter，并把非 Clifford rotation synthesis 成本纳入资源模型。若暂时做不到，则论文必须严格限定为 X/CNOT/MCT bit-flip oracle 口径。第二，在当前 ROS-style LUT proxy 的基础上继续接入官方 ROS、mockturtle 和 legacy RevKit/CirKit 流程，把比较从 proxy/API lower bound 扩展到更标准的外部流程。第三，large frontier policy 和 cost-aware frontier policy 已经把结构级 AI 从单一质量追求推进到可调质量-时间折中，但还可以继续把辅助线生命周期、T-depth proxy 和 score 统一进多目标 teacher 或 Pareto policy。完成这些以后，论文就更接近“方法贡献”而不是“阶段性项目报告”。

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这一步看似只是把经典逻辑接入量子程序，但在容错量子计算中，它往往决定了非 Clifford 资源和辅助比特压力。若直接按 ANF 单项式逐项实现，线路语义透明，但容易重复计算相似结构；若只优化 CNOT 或只压缩某一种图表示，又可能把成本转移到 T-count、深度或辅助比特上。因此，本文关注的问题不是“如何复现某个已有 CNOT 优化算法”，而是“如何把 Boolean oracle synthesis 写成一个可由 AI 辅助的多资源搜索问题”。

本文采用 ANF 作为语义基准：

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

其中 $\mathcal{M}$ 是需要综合的 square-free monomial 集合。该表示有两个优势。第一，任一候选线路都可以回到 GF(2) 多项式进行等价验证。第二，搜索动作可以直接定义为对项集合的代数变换，例如公共因子提取、FPRM 极性重写、线性奇偶因子筛选和递归子问题分解。它的困难也很明确：原始 ANF 不显式给出共享结构，动作空间随项数和变量数迅速增长，必须用搜索策略、神经先验和保守兜底共同控制。

本文的一句话论证是：在逻辑层量子布尔函数 oracle 综合中，把 ANF/FPRM 项集合当作搜索状态，并用资源感知神经搜索选择可计算、可反算的代数分解动作，可以在传统函数和高维项集合上降低 T-count 与加权逻辑资源；其边界是当前结果仍停留在逻辑层，不包含 routing、magic-state factory、真实噪声模型或硬件后端映射。

本文贡献概括如下：

- 提出独立于 SSHR 的 Resource-NMCTS 搜索表述，以 ANF/FPRM 项集合为状态，以逻辑资源 score 为目标，把 Boolean oracle synthesis 写成可验证的组合搜索问题。

- 引入 Boolean-ring screen, 在 square-free Boolean ring 中允许线性因子与 quotient 共享变量, 从而扩大高维可搜索因子结构。
- 将 AI 模块分为动作排序、结构深度选择、depth-frontier policy 和 baseline-preserving guard, 避免把学习策略写成不可控的黑箱生成器。
- 在 $n \leq 6$ 传统函数、 $n = 18, 20$ 函数级边界、 $n = 20$ 到 40 项集级 scale、 $n = 21, 22, 23$ 完整 truth-table bridge 以及 emitted-circuit schedule proxy 上给出资源、正确性和逻辑层时序证据。

2 相关工作与本文定位

2.1 布尔表示驱动的 oracle 综合

Xor-And-Inverter Graphs (XAG) 把 XOR 和 AND 结构分开, 使 AND 节点数量与低 T-count oracle 编译直接相关。Meuli 等人讨论了 multiplicative complexity 在低 T-count oracle 编译中的作用 [1], 并将 XAG 用于量子编译 [2]。ROS 将 oracle synthesis 表述为 resource-constrained LUT mapping [3], 进一步说明布尔表示选择会影响 T-count、辅助比特和线路深度。BDD-based reversible synthesis 则展示了面向大函数的可扩展布尔表示路径 [4]。后端感知 oracle synthesis 开始把逻辑综合和 fault-tolerant back-end 指标连接起来 [5]。

本文与上述工作的共同点是承认布尔中间表示决定逻辑资源; 差异在于本文不固定使用 XAG、LUT 或 BDD, 而是在 ANF/FPRM 项集合上显式搜索可反算的因子结构。这样做的优点是语义验证直接, 代价是候选动作空间更大, 需要 AI 和 guard 控制搜索。

SSHR 利用 parallelotope 的空间结构做 small-scale Boolean function 的 CNOT-oriented synthesis [8]。本文不使用 SSHR 的候选空间, 也不把论文目标设为 CNOT-only 最优。SSHR 在本文中是一个重要但定位明确的 baseline: 它帮助说明本文在低 T-count 和加权 score 上的优势, 也提醒我们在 CNOT 指标上必须诚实报告 trade-off。

2.2 学习式搜索与线路优化

学习式搜索已经出现在经典布尔电路设计、量子线路综合和 ZX-calculus 优化中。ShortCircuit 使用 AlphaZero 风格搜索进行经典布尔电路设计 [9]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis [10]; 强化学习和图神经网络也被用于 ZX-calculus rewrite-rule 选择 [11]; MonteQ 使用 MCTS 处理量子线路综合中的动作排序问题 [12]。这些工作证明, 离散线路综合中的动作选择可以受益于学习策略。

本文的区别在于状态不是任意量子门序列, 也不是 ZX 图, 而是 Boolean oracle 的 ANF/FPRM 项集合。神经模型不直接输出线路, 而是排序或选择候选代数分解动作; 最终线路仍由可验证的 compute/action/uncompute 结构生成。这一设计降低了 AI 生成错误线路的风险, 也使每个候选都可以经过 ANF 展开、GF(2) 符号模拟或完整 truth-table 检查。

3 问题定义与资源模型

本文输入是布尔函数的 truth-table 或 ANF 项集合，输出是实现式 (1) 的逻辑层可逆线路。线路由 X、CNOT 和多控 Toffoli (MCT) 抽象门组成，并在资源统计中估计 T-count、CNOT、depth、gate count 和 peak ancilla。搜索排序使用统一分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数不是物理设备的时间或错误率模型，而是逻辑层候选排序目标。为避免单一分数掩盖 trade-off，实验同时报告各个资源分量。

正确性验证分三层。第一，在函数级小规模和边界切片中构造完整 truth-table，逐点检查 oracle 输出。第二，在大规模项集实验中，将 factor plan 展开回 ANF 项集合，检查目标多项式是否等价。第三，对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟，检查输出线多项式、输入线复原和辅助线清零。第三层验证不等同于 2^n 个输入的完整枚举，但它检查的是最终线路而不只是搜索 plan，因而强于只报告资源估计。

表 1: 本文核心术语。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架，包括神经动作先验、MCTS、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支，允许线性因子与 quotient 共享变量。
Depth policy	在 single、depth-1 和 depth-2 screen 之间选择的结构级神经策略。
Depth-frontier policy	在 depth-2、depth-3 和 depth-4 screen 之间选择的高预算质量-时间折中策略。
Guard	以确定性 baseline 为兜底的保守门控，只有当学习候选不劣于 baseline 时才接受。
Truth-table bridge	在 $n = 21, 22, 23$ 构造完整 truth-table，把大规模项集符号验证和函数级完整验证连接起来。
SSHR	CNOT-oriented small-scale baseline；本文方法不使用 SSHR parallelotope 候选。

4 方法

4.1 搜索状态、动作与线路生成

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构，将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题，再递归生成 compute/action/uncompute 形式的线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子，以及由神经模型扩展的 root actions。

动作 a 被接受后，框架会估计其即时资源收益、子问题大小、辅助比特峰值和后续可分解性。神经动作先验对候选排序，MCTS 在有限预算内探索候选组合，rollout 使用确定性资源估计和已

有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选取最优者。该过程的关键不是让 AI 直接创造量子门，而是让 AI 在大量可验证的代数动作中更早访问有资源收益的分支。

4.2 布尔环线性因子

传统线性因子筛选容易要求线性因子变量和 quotient 变量不相交。这个限制便于实现，但会排除 square-free Boolean ring 中合法的重叠结构。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (4)$$

线路层面仍可用 CNOT 计算线性奇偶量，再以该辅助量控制 quotient 子线路。这样，Boolean-ring screen 不是简单增加 beam 宽度，而是把原先被实现约束排除的代数候选重新放回可综合空间。

4.3 神经策略、结构策略与保守 guard

本文把 AI 的角色拆成三类。第一类是动作排序：模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择：depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 Boolean-ring screen。第三类是 frontier 选择：depth-frontier policy 在 depth-2、depth-3 和 depth-4 Boolean-ring screen 之间选择。质量型 frontier policy 用 score-only oracle frontier 训练，目标是在接近 depth-4 或 all-depth frontier 的质量时减少全深度枚举开销；cost-aware frontier policy 则在标签中加入相对运行时间惩罚，用较小 score 牺牲换取更短 plan time 和更低辅助线生命周期。

保守 guard 用来限制学习策略的风险。若学习候选相对确定性 baseline 出现 score 退化，则返回 baseline。Depth-2 skip guard 采用 zero-false-skip 阈值，优先避免把应该运行 depth-2 的项集合错误跳过。Screen-gate 则用于判断在某些边界函数上是否可以跳过昂贵的 Resource tail。这样的设计降低了策略模型分布外失误对资源结果的影响，但也意味着部分 AI 贡献表现为运行时间节省或质量-时间折中，而不是无条件的质量支配。

表 2: Resource-NMCTS 的模块与证据钩子。

模块	机制	主要证据类型
ANF/FPRM 搜索状态	把 oracle 综合写成项集合分解问题，所有候选可回到 GF(2) 多项式验证。	小规模 truth-table 验证与项集级 ANF 验证。
Boolean-ring screen	搜索允许共享变量的线性因子，递归处理 quotient/rest 子问题。	$n = 18, 20$ 函数级边界和 $n = 20-40$ scale。
神经动作先验与 MCTS	对候选动作排序并在有限预算内组合分解动作。	传统函数上的搜索消融和 portfolio 改善。
Depth policy / guard	学习 screen 深度或跳过较深搜索，并用 baseline-preserving 规则兜底。	held-out 项集时间节省和无 score-loss guard。
Depth-frontier policy	学习高预算 depth-2/3/4 质量前沿，并提供 score-only 与 cost-aware 两种运行模式。	$n = 20, 28, 40$ scale、独立 seed 泛化与 $n = 21, 22, 23$ bridge。

5 实验设计

实验分为七个层次。第一层是 $n \leq 6$ 传统函数，用 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、ABC/BDD 估计、ROS-style LUT proxy、RevKit Python `oracle_synth` 和 SSHR-H/SSHR-I 作为 baseline，主要回答小规模函数上是否形成低 T-count 和低加权资源优势，以及该优势在 RevKit Rz-phase lower-bound 口径下暴露出什么差距。第二层是 $n = 14$ RevKit API 隔离超时探针，对高维 truth-table 输入逐行用子进程硬超时运行，主要回答 RevKit API baseline 在高维上是否能形成 paired benchmark。第三层是 $n = 14, 15, 16, 18$ 的高维外部导出集，对 ABC-AIG/XAG/LUT/BDD 和 ROS-style LUT proxy 做统一 truth-table 校验，主要回答外部布尔表示路线在高维上是否会削弱本文优势。第四层是 $n = 18, 20$ 函数级边界，完整构造 truth-table 并验证 emitted oracle circuit，主要回答高维函数上哪些搜索分支仍可运行。第五层是 $n = 20$ 到 40 项集级 scale，不构造完整 truth-table，而是对 plan 和 emitted circuit 做符号等价验证，主要回答结构策略是否可扩展，并通过独立 seed 的 $n = 24, 28, 32, 40$ 泛化集检查 frontier policy、large frontier policy 和 cost-aware frontier policy 的稳定性。第六层是 $n = 21, 22, 23$ truth-table bridge，对一小组生成式 ANF 函数重新构造完整 truth-table，专门检查项集级符号验证是否与函数级 oracle 验证一致。第七层是 emitted-circuit schedule proxy，在不做硬件 mapping 的前提下估计并行逻辑深度、CNOT-depth proxy、T-depth proxy 和显式辅助线 lifetime area。

所有 paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行保留为运行边界证据，但不参与正确结果均值比较。所有结论限定在逻辑层资源，不声称硬件 mapping、连通性约束或噪声模型下最优。

6 结果

6.1 传统函数上的低 T-count 与低 score 优势

表 3 给出 $n \leq 6$ 传统函数上的平均资源。相对 direct ANF，Pareto-Resource-NMCTS 平均 T-count 降低 72.25%，平均 score 降低 67.80%。相对 ESOP cube beam，Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%。相对 weighted ESOP-MILP，获得 167/3/7，平均 score 降低 29.84%。SSHR-H 的平均 CNOT 最低，因此本文不能主张 CNOT-only 支配；更准确的结论是，本文用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低加权 score。

表 3: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

6.2 ROS-style LUT proxy 外部对比

官方 ROS 把 oracle synthesis 写成 resource-constrained LUT mapping, 并配合垃圾管理策略降低辅助线压力。当前环境尚不能直接运行官方 ROS/mockturtle 或 legacy RevKit/CirKit 流程; RevKit Python API 的单独结果见下一节。因此本文新增一个明确标注的 ROS-style LUT proxy: 对同一导出 BLIF benchmark 运行 ABC ‘if -K’, 在 $K = 3, 4, 5$ 之间做 sweep; 每个映射后的 LUT 网络按 local ANF compute/action/uncompute 方式估计 X/CNOT/MCT 逻辑资源, 再按式 (3) 选择 best- K 。这不是官方 ROS 复现, 也没有 SAT garbage management, 但它比单个固定 $K = 4$ 的 ABC-LUT baseline 更强, 更适合作为 LUT 路线的压力测试。

该 proxy 覆盖 $n = 3-6$ 的 177 个传统函数, 以及 $n = 14, 15, 16, 18$ 的 132 个高维导出函数, 共 309 个函数。三档 K sweep 产生 927 个外部映射行, 全部通过 truth-table 检查, 没有 timeout、error 或 incorrect 行。Best- K proxy 相对固定 $K = 4$ 自身获得 219/0/90 的 score 胜/负/平, 平均 score 降低 18.12%; 因此它确实是比原 ABC-LUT 更强的 LUT baseline。在这个更强 baseline 上, Resource-NMCTS 仍在 309/309 个函数上取得更低 score, 平均 score 降低 83.77%; Pareto-Resource-NMCTS 为 309/0/0、平均 score 降低 84.27%。高维 profile variant 在 132 个高维函数上相对该 proxy 为 132/0/0、平均 score 降低 97.38%。

表 4: LUT proxy 外部对比。该 proxy 不是官方 ROS/mockturtle 或 legacy RevKit/CirKit 复现。

比较	指标	函数数	胜/负/平	平均 Δ
Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-83.77%
Profile-Resource-NMCTS vs ROS-style LUT proxy	score	132	132/0/0	-97.38%
Pareto-Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-84.27%
AND-direct ANF vs ROS-style LUT proxy	score	309	307/2/0	-67.45%
ROS-style LUT proxy vs fixed $K = 4$ LUT	score	309	219/0/90	-18.12%

6.3 RevKit API oracle synthesis 与 R_z 角度审计

在 toolchain readiness 审计基础上, 本文进一步在本地 mcts-qoracle 环境中安装 RevKit Python API, 并调用其 `oracle_synth` 对 $n \leq 6$ 的 177 个完整 truth-table 传统函数逐一合成。该流程不是 ABC proxy: 输入为完整 truth table, 输出为 RevKit 的 R_z -phase netlist; 本文按返回 netlist 统计 T/Tdg 或奇数倍 $R_z(\pi/4)$ 、CX、qubit-conflict depth、gate 数、额外 qubit 数, 并额外审计所有 R_z 角度是否属于 Clifford+T 可直接计数的 $\pi/4$ 网格。这仍不是硬件 mapping, 也不是 legacy CirKit CLI, 但已经是一个真实 RevKit API baseline。

表 5 显示, 这个 baseline 给出一个必须正视的外部边界。在当前 X/CNOT/MCT bit-flip 资源口径下, Resource-NMCTS 对 ESOP、ABC-LUT proxy 和 direct ANF 有明显优势; 但在 RevKit 的 R_z -phase netlist lower-bound 口径下, Resource-NMCTS 相对 RevKit 的 score 为 6/171/0, 平均高 751.69%, T-like count 平均高 4060.08%。同时, 角度审计显示 177 行中只有 6 行不含非 Clifford R_z ; 171 行含非 Clifford R_z , 总数为 9242, 角度除以 π 的最大分母为 64。因此, 这组数字不能写成精确 Clifford+T T-count 比较。更准确的解释是: RevKit API 暴露了一个强 phase-rotation lower-bound baseline, 而本文当前 emitter 固定在 X/CNOT/MCT compute/action/uncompute 路线。进一步的符号敏感性检查显示, 若每个非 Clifford R_z 只计 1 个 score 单位, Resource-NMCTS 与 Pareto-Resource-NMCTS 已分别转为 140/37/0 和 157/20/0; 若计 2 个 score 单位, 二者均为

177/0/0。这说明差距主要由 phase rotation 成本口径决定，而不是简单的搜索失败。投稿前要么把论文主张严格限定在 bit-flip oracle 逻辑层，要么新增 phase/Rz-aware emitter，并把非 Clifford rotation synthesis 或近似成本纳入资源模型。

表 5: RevKit Python `oracle_synth` Rz-phase lower-bound baseline 与符号 Rz 成本敏感性。171/177 行含非 Clifford R_z ，因此 lower-bound score 与 T-like count 不是精确 Clifford+T 成本；score+1/Rz 与 score+2/Rz 仅用于敏感性分析。

比较	指标	数量	胜/负/平	平均 Δ
Resource-NMCTS vs RevKit <code>oracle_synth</code>	lower-bound score	177	6/171/0	+751.69%
Resource-NMCTS vs RevKit <code>oracle_synth</code>	score+1/Rz	177	140/37/0	-14.52%
Resource-NMCTS vs RevKit <code>oracle_synth</code>	score+2/Rz	177	177/0/0	-53.48%
Pareto-Resource-NMCTS vs RevKit <code>oracle_synth</code>	score+1/Rz	177	157/20/0	-17.89%
Pareto-Resource-NMCTS vs RevKit <code>oracle_synth</code>	score+2/Rz	177	177/0/0	-55.24%
Resource-NMCTS vs RevKit <code>oracle_synth</code>	T+Rz	177	148/18/11	-23.88%

Phase/Rz 成本感知 portfolio. 上述单方法比较仍没有利用本文已有的 portfolio 结构。为避免把 RevKit lower-bound 结果误读成“搜索路线失败”，本文进一步运行一个可复现的 phase/Rz 成本敏感性分析：对每个函数，分别在 Resource-NMCTS family (Resource-NMCTS、Pareto-Resource-NMCTS、FPRM polarity archive)、传统 baseline family (direct ANF、AND-direct ANF、cube beam、ESOP-MILP、SSHR-H) 和 all-internal family 内选择普通 score 最低的已验证 bit-flip circuit，然后把 RevKit baseline 的每个非 Clifford R_z 额外计为 λ_{R_z} 个 score 单位。这个 reranker 不改变线路语义，也不是精确 rotation synthesis；它只回答在何种 phase-cost 口径下当前 bit-flip 资源路线重新变得有竞争力。

表 6 显示，Resource-NMCTS family 的中位 break-even 为 0.80/Rz，winner 分布为 Pareto-Resource-NMCTS 115 行、FPRM polarity archive 62 行。 $\lambda_{R_z} = 1$ 时，该 family 已相对 RevKit 获得 157/20/0，平均 score 降低 17.89%； $\lambda_{R_z} = 1.5$ 时达到 177/0/0。相比之下，传统 baseline family 在 $\lambda_{R_z} = 1$ 时仍为 80/97/0，说明 Resource-NMCTS family 的相对优势不是由任意传统方法组合自动得到，而是来自当前资源搜索与 FPRM polarity archive 的互补。这个结果把下一步目标量化得更清楚：若要扩展到 phase/Rz-aware emitter，需要重点降低或显式综合 RevKit 暴露出的非 Clifford rotation，而不是继续只优化 bit-flip MCT 线路。

表 6: Phase/Rz 成本敏感性下的 score-reranked portfolio。 λ_{R_z} 表示每个 RevKit 非 Clifford R_z 额外计入的 score 单位；Covered 为 break-even threshold 不超过当前 λ_{R_z} 的函数数。

Portfolio	λ_{R_z}	Items	W/L/T	Mean Δ	Covered
Resource-NMCTS family	0	177	6/171/0	+711.60%	6/177
Resource-NMCTS family	1	177	157/20/0	-17.89%	157/177
Resource-NMCTS family	1.5	177	177/0/0	-42.26%	177/177
Resource-NMCTS family	2	177	177/0/0	-55.24%	177/177
Traditional baseline family	1	177	80/97/0	+7.58%	80/177
Traditional baseline family	1.5	177	160/17/0	-24.59%	160/177
Traditional baseline family	2	177	176/1/0	-41.71%	176/177
All-internal score portfolio	1	177	157/20/0	-17.96%	157/177
All-internal score portfolio	1.5	177	177/0/0	-42.31%	177/177
All-internal score portfolio	2	177	177/0/0	-55.28%	177/177

近似 **Clifford+T** 旋转综合成本模型。符号 λ_{R_z} 只能说明 break-even 位置，不能回答容错 Clifford+T 口径下 RevKit phase netlist 的成本量级。为此，本文进一步加入一个近似 rotation synthesis proxy。根据 Ross-Selinger 的 Z 旋转近似综合结果，典型 T -count 可按 $3\log_2(1/\epsilon)$ 量级估计 [6]；Selinger 的一般单量子比特近似算法给出更保守的 $4\log_2(1/\epsilon) + K$ 型 proxy [7]。本文不生成具体 Clifford+T 序列，而是把 RevKit 每个非 Clifford R_z 额外计入

$$T/R_z = \lceil a \log_2(1/\epsilon) + b \rceil$$

个 T gates，并同步增加 gate 和 depth proxy。表 7 显示，即使取较宽松的 Ross-Selinger-style $\epsilon = 10^{-3}$ ，每个非 Clifford R_z 也对应 30 个 T gates；由于 RevKit 在 177 个传统函数中累计有 9242 个非 Clifford R_z ，其 synthesized-cost proxy 平均 score 大幅上升。在这个口径下，Resource-NMCTS family 和传统 baseline family 都相对 RevKit 获得 177/0/0；二者差别不再是能否击败 RevKit，而是内部 family 之间的低资源优先级。这个结果的正确解释是：RevKit lower-bound phase netlist 不能直接当作 Clifford+T 优势；若目标论文仍限定 Clifford+T 逻辑层，则必须要么加入真实 rotation synthesis，要么把 RevKit phase baseline 单独列为不同 gate-set 口径。

表 7: 近似 Clifford+T R_z 综合成本模型下的 RevKit 对比。该表只使用 $T/R_z = \lceil a \log_2(1/\epsilon) + b \rceil$ 的逻辑层 proxy，不输出具体 Clifford+T 旋转序列，也不包含硬件 mapping。

Portfolio	Model/ ϵ	T/Rz	Items	W/L/T	Mean Δ
Resource-NMCTS family	ross_selinger_z/1e-03	30	177	177/0/0	-95.03%
Resource-NMCTS family	ross_selinger_z/1e-06	60	177	177/0/0	-96.51%
Resource-NMCTS family	ross_selinger_z/1e-10	100	177	177/0/0	-97.11%
Resource-NMCTS family	selinger_su2/1e-06	90	177	177/0/0	-97.01%
Traditional baseline family	ross_selinger_z/1e-06	60	177	177/0/0	-96.05%

高维 RevKit API 隔离超时探针。 上述 RevKit paired comparison 只覆盖 $n \leq 6$ ，原因不只是运行预算，而是 API 层可扩展性本身需要审计。为避免人工中止带来的不可复现性，本文新增一个隔离超时探针：对 $n = 14$ 高维函数逐行启动一次 RevKit `oracle_synth` 子进程，达到 30 s wall-time cutoff 后直接终止该子进程，并把 timeout 行保留为运行边界而不是资源比较行。表 8 显示，在前 8 个 $n = 14$ 函数中，RevKit 仅 1 行返回，7 行触发 30 s timeout。

唯一返回的函数为 `anf_n14_10`。RevKit lower-bound phase netlist 的 score 为 2948.79，而 Resource-NMCTS 的普通 score 为 10358.47，因此在未计非 Clifford phase cost 的 lower-bound 口径下 Resource-NMCTS 为 loss；但该 RevKit netlist 含 32767 个非 Clifford R_z ，若仅加入 $1/R_z$ 的符号成本，RevKit score 变为 35715.79，Resource-NMCTS 转为 win，relative score 为 -71.00%。这个结果说明两点：第一，高维 RevKit API 不能在当前适配器下作为普通 paired benchmark 批量纳入均值；第二，即使返回结果，其解释仍受 phase/ R_z gate-set 口径支配。因此，高维 RevKit 结果在本文中只作为可扩展性和 gate-set 边界证据，而不作为 Resource-NMCTS 的高维胜负统计。

表 8: $n = 14$ RevKit Python `oracle_synth` 隔离超时探针。Timeout 行没有线路资源，不能参与资源均值比较。

n	Items	Usable	Timeouts	Other errors	Median time (s)
14	8	1	7	0	30.00

6.4 高维函数级边界

在 $n = 18$ 的 12 个随机 ANF 函数上，adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1，平均 score 从 11349.06 降至 10670.94；完整 Resource-NMCTS 平均 score 为 7315.62，相对 adaptive screen 进一步降低 23.85%。这说明中等高维上 screen 是强候选生成器，但不能完全替代 Resource tail。

在 $n = 20$ 边界函数上，FPRM root beam 和 fast linear-pair 均触发 300 s task timeout。Adaptive depth-2 screen 相对 single screen 获得 5/0/1，平均 score 降低 7.47%；集成该分支后，Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1，平均 score 降低 11.80%。该切片中完整 Resource tail 与 adaptive screen 资源持平，因此 screen-gated Resource-NMCTS 可把平均时间从 77.10 s 降至 20.71 s。这个结果支持运行时门控，但也提示 $n = 20$ 切片的主要质量收益来自结构筛选而非深层 MCTS tail。

表 9: $n = 18, 20$ 函数级边界结果。

切片	方法	平均 T	平均 CNOT	平均 score	平均时间 (s)
$n = 18$	Single screen	10389.33	16268.42	11349.06	0.82
$n = 18$	Adaptive depth-2 screen	9767.00	15301.58	10670.94	4.39
$n = 18$	Resource-NMCTS	6639.33	11380.92	7315.62	226.18
$n = 20$	AND-direct ANF	21516.67	33635.67	23491.15	10.41
$n = 20$	Single screen	20786.00	32492.50	22695.15	10.70
$n = 20$	Adaptive depth-2 screen	19535.33	30542.50	21331.24	20.67
$n = 20$	Resource-NMCTS	19535.33	30542.50	21331.24	77.10
$n = 20$	Screen-gated Resource-NMCTS	19535.33	30542.50	21331.24	20.71

6.5 结构级 AI 与高预算 frontier

Depth policy 在 $n = 14, 16, 18$ 项集上训练，并在 held-out $n = 20$ 项集上评估。Policy 相对 single screen 获得 42/0/6，平均 score 降低 6.57%；相对 all-depth adaptive 平均 score 只高 0.28%，但平均运行时间降低 34.71%。保守 depth-2 skip guard 在 held-out $n = 20$ test 中没有 false skip，

96 个样本全部与 fixed depth-2 screen score 持平，并相对 all-depth adaptive 节省 24.74% 平均时间。

为判断 fixed depth-2 是否已经达到质量上限，本文在 $n = 20, 28, 40$ 上运行 depth-3/4 Boolean-ring screen。Depth-4 相对 fixed depth-2 获得 49/0/23，平均 score 降低 3.10%，但运行时间增加 681.55%。因此，depth-3/4 是高预算质量模式，而不是默认快速模式。Depth-frontier policy 将这一质量前沿学习化：在 $n = 20, 28, 40$ scale harness 中，它相对 fixed depth-2 获得 35/0/37，平均 score 降低 2.19%；相对 all-depth adaptive depth ≤ 4 平均 score 只高 0.97%，但节省 58.69% 时间。独立 seed 的 $n = 24, 28, 32, 40$ 泛化集进一步显示，frontier policy 相对 fixed depth-2 获得 40/0/56，平均 score 降低 1.85%；相对 fixed depth-4 只高 0.61% score，并节省 23.40% plan time。

为了补强“AI 结构策略质量收益仍偏弱”的问题，本文进一步扩大 depth-frontier policy 的训练集：训练样本从 96 增至 192，验证样本从 36 增至 72，held-out 测试样本从 32 增至 48，并把隐藏层宽度从 96 提高到 160。Large frontier policy 在 held-out 测试上相对 oracle depth-2/3/4 frontier 的平均 score gap 从旧模型的 0.80% 降至 0.04%，同时相对 all-depth frontier evaluation 节省 51.30% 时间。在同一独立 seed 的 $n = 24, 28, 32, 40$ 泛化集上，large frontier policy 相对 fixed depth-2 获得 56/0/40，平均 score 降低 2.34%；相对旧 frontier policy 为 17/0/79，平均 score 再降 0.49%；相对 all-depth adaptive 只高 0.10%，仍节省 53.50% 时间。该结果说明结构级 AI 的质量 gap 可以通过更充分的 frontier teacher 数据显著压缩，但它也更频繁选择 depth-3/4，因此比旧 policy 更慢。

进一步地，本文把 frontier policy 的标签从 score-only oracle 扩展为 cost-aware oracle。当前 cost-aware 版本使用 $score_{\Delta} + 0.003 time_{\Delta}$ 作为标签目标，仍在相同的 192/72/48 项集规模和 hidden=160 模型上训练。Held-out 测试中，该策略相对 cost-aware frontier 的平均 score gap 为 0.38%，但相对 all-depth frontier evaluation 节省 71.54% 时间。在独立 seed 的 $n = 24, 28, 32, 40$ 泛化集上，它相对 fixed depth-2 仍为 56/0/40，平均 score 降低 1.39%，同时把 plan-time 增幅控制在 170.03%；相对 large policy，它牺牲 0.99% 平均 score，但减少 33.02% plan time 和 7.61% 显式辅助线 lifetime area。因此，frontier policy 现在不只是单一模型，而是形成了“质量模式”和“快速质量模式”两个可选运行点。

表 10: 结构级策略与 depth-frontier 结果。

设置	比较	score 胜/负/平	平均 Δ score	平均 Δ time
held-out $n = 20$	depth policy vs single screen	42/0/6	-6.57%	+2328.66%
held-out $n = 20$	depth policy vs all-depth adaptive	0/6/42	+0.28%	-34.71%
held-out $n = 20$	depth-2 guard vs fixed depth-2	0/0/96	+0.00%	-0.54%
scale $n = 20, 28, 40$	screen depth-4 vs fixed depth-2	49/0/23	-3.10%	+681.55%
scale $n = 20, 28, 40$	frontier policy vs fixed depth-2	35/0/37	-2.19%	+503.20%
scale $n = 20, 28, 40$	frontier policy vs all-depth	0/16/56	+0.97%	-58.69%
generalization $n = 24, 28, 32, 40$	frontier policy vs fixed depth-2	40/0/56	-1.85%	+456.43%
generalization $n = 24, 28, 32, 40$	frontier policy vs depth-4	0/19/77	+0.61%	-23.40%
large held-out	large frontier vs oracle frontier	0/3/45	+0.04%	-51.30%
large generalization	large frontier vs fixed depth-2	56/0/40	-2.34%	+563.80%
large generalization	large frontier vs old frontier	17/0/79	-0.49%	+119.26%
large generalization	large frontier vs all-depth	0/6/90	+0.10%	-53.50%
cost held-out	cost frontier vs cost-aware frontier	2/6/40	+0.38%	-71.54%
cost generalization	cost frontier vs fixed depth-2	56/0/40	-1.39%	+170.03%
cost generalization	cost frontier vs large frontier	1/48/47	+0.99%	-33.02%
cost generalization	cost frontier vs all-depth	0/53/43	+1.10%	-76.54%

6.6 项集级 scale 与完整验证桥接

项集级 scale 覆盖 $n = 20, 22, 24, 28, 32, 36, 40$, 主 scale、extended scale、depth-frontier、frontier-policy rerun、独立泛化 run、large frontier policy 泛化 run 和 cost-aware frontier policy 泛化 run 合计 6600 个方法行。所有方法行均通过两层验证: factor plan 的 ANF 展开验证为 6600/6600, emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证为 6600/6600, mismatch 总数为 0。该结果说明大规模项集资源表不是只统计 plan, 但仍需承认它不是完整 truth-table 枚举。

Truth-table bridge 进一步把完整验证推进到 $n = 21, 22, 23$ 。在 18 个生成式 ANF 函数和 10 个方法上, 180/180 个方法行通过完整 truth-table oracle 验证、plan ANF 展开验证和 emitted-circuit GF(2) 符号验证, mismatch 为 0。Large frontier policy 和 cost-aware frontier policy 又在相同 seed 的 $n = 23$ 六函数切片上各重新运行 60 个方法行, 也全部通过完整 truth-table oracle、plan 和 emitted-circuit 验证。因此, 当前 bridge 证据合计为 300/300 个方法行通过三层验证。其中原 $n = 23$ bridge 平均 truth-table 构造时间为 209.54 s/function, large rerun 为 208.61 s/function, cost-aware rerun 为 209.42 s/function。该 bridge 还复现了 frontier 信号: 原 frontier policy 在 $n = 23$ 上相对 fixed depth-2 获得 4/0/2, 平均 score 降低 1.88%; large frontier policy 提升为 5/0/1, 平均 score 降低 2.36%; cost-aware frontier policy 为 4/0/2, 平均 score 降低 1.46%, 但 plan time 增幅仅为 196.09%。相对 large frontier policy, cost-aware frontier policy 以 0.92% 平均 score 代价换取 56.29% plan time 下降和 12.62% lifetime area 下降。

表 11: 大规模项集验证与 truth-table bridge。

验证项或比较	结果	说明
项集级 plan ANF 验证	6600/6600	覆盖 $n = 20$ 到 40 的 scale/frontier/frontier-policy rerun、独立 seed 泛化 run、large policy 泛化 run 和 cost-aware policy 泛化 run。
项集级 emitted-circuit 符号验证	6600/6600	GF(2) 多项式模拟通过, mismatch 总数为 0。
$n = 21, 22, 23$ 、large $n = 23$ 与 cost-aware $n = 23$ 完整 truth-table oracle 验证	300/300	18 个原 bridge 函数加 12 个 $n=23$ rerun 函数, 所有 emitted circuit 逐点验证通过。
$n = 21, 22$ frontier policy vs fixed depth-2	8/0/4	平均 score -3.50% , plan time $+634.71\%$ 。
$n = 23$ frontier policy vs fixed depth-2	4/0/2	平均 score -1.88% , plan time $+759.95\%$ 。
$n = 23$ frontier policy vs all-depth	0/1/5	平均 score $+0.61\%$, plan time -48.77% 。
large $n = 23$ frontier vs fixed depth-2	5/0/1	平均 score -2.36% , plan time $+790.62\%$ 。
large $n = 23$ frontier vs old frontier	1/0/5	平均 score -0.48% , T-depth proxy -0.45% 。
cost-aware $n = 23$ frontier vs fixed depth-2	4/0/2	平均 score -1.46% , plan time $+196.09\%$ 。
cost-aware $n = 23$ frontier vs large frontier	0/5/1	平均 score $+0.92\%$, plan time -56.29% 。

6.7 逻辑层 schedule proxy 与辅助线生命周期

为避免只报告静态资源总量, 本文进一步在 emitted X/CNOT/MCT oracle circuit 上计算逻辑层 schedule proxy。该分析不做硬件 mapping, 也不使用连通性、routing、噪声或 magic-state factory 模型; 它只把每个 emitted gate 按线冲突做并行层调度, 并用 MCT 分解的 CNOT 数和 T-stage proxy 估计 CNOT-depth 与 T-depth, 同时记录显式辅助线从第一次使用到最后一次使用的 lifetime area。因此, 这组指标应理解为后端相关的逻辑层代理指标, 而不是物理设备运行时间。

表 12 显示, depth-frontier policy 的质量收益在 T-depth proxy 上也成立。在独立 seed 的 $n = 24, 28, 32, 40$ 泛化集中, frontier policy 相对 fixed depth-2 获得 40/0/56 的 score 胜/负/平, 平

均 score 降低 1.85%；同一比较下 T-depth proxy 也为 40/0/56，平均降低 1.85%。Large frontier policy 在同一泛化集上把 score 提升为 56/0/40、平均降低 2.34%，T-depth proxy 平均降低 2.28%，但显式辅助线 lifetime area 增幅扩大到 26.13%。Cost-aware frontier policy 保持 56/0/40 的 score 胜/负/平，平均 score 降低 1.39%，T-depth proxy 平均降低 1.36%，同时把 lifetime area 增幅压到 14.63%。

在 $n = 21, 22$ 完整 truth-table bridge 中，frontier policy 相对 fixed depth-2 的 T-depth proxy 为 8/0/4、平均降低 3.32%；在新增 $n = 23$ bridge 中为 4/0/2、平均降低 1.69%。Large frontier policy 在 $n = 23$ bridge 中相对 fixed depth-2 的 T-depth proxy 平均降低 2.14%，相对旧 policy 再降低 0.45%，但辅助线 lifetime area 相对旧 policy 增加 4.00%。Cost-aware frontier policy 在 $n = 23$ bridge 中相对 fixed depth-2 的 T-depth proxy 平均降低 1.43%，相对 large policy 的 T-depth proxy 高 0.73%，但 plan time 和 lifetime area 分别降低 56.29% 和 12.62%。因此，新策略组应写成质量增强型和快速质量型 frontier policy，而不是单一“更优”策略。

表 12: 逻辑层 schedule proxy 结果。所有指标来自 emitted circuit，不包含硬件 mapping。

设置	对比	score	T-depth proxy	aux area
$n = 24, 28, 32, 40$ schedule 泛化	frontier policy vs fixed depth-2	40/0/56, -1.85%	40/0/56, -1.85%	+20.09%
$n = 24, 28, 32, 40$ schedule 泛化	frontier policy vs depth-4	0/19/77, +0.61%	0/19/77, +0.55%	-5.42%
$n = 21, 22$ schedule bridge	frontier policy vs fixed depth-2	8/0/4, -3.50%	8/0/4, -3.32%	+32.93%
$n = 21, 22$ schedule bridge	frontier policy vs depth-4	0/2/10, +0.32%	0/2/10, +0.32%	-4.01%
$n = 23$ schedule bridge	frontier policy vs fixed depth-2	4/0/2, -1.88%	4/0/2, -1.69%	+29.49%
$n = 23$ schedule bridge	frontier policy vs depth-4	0/1/5, +0.61%	0/1/5, +0.52%	-5.09%
large $n = 24, 28, 32, 40$	large frontier vs fixed depth-2	56/0/40, -2.34%	-2.28%	+26.13%
large $n = 24, 28, 32, 40$	large frontier vs all-depth	0/6/90, +0.10%	+0.10%	-1.17%
large $n = 23$ bridge	large frontier vs old frontier	1/0/5, -0.48%	-0.45%	+4.00%
cost $n = 24, 28, 32, 40$	cost frontier vs fixed depth-2	56/0/40, -1.39%	-1.36%	+14.63%
cost $n = 24, 28, 32, 40$	cost frontier vs large frontier	1/48/47, +0.99%	+0.97%	-7.61%
cost $n = 23$ bridge	cost frontier vs large frontier	0/5/1, +0.92%	+0.73%	-12.62%

7 证据地图与边界

表 13 将当前可写入论文的主张与证据对应起来。最重要的写法是把“AI”讲成可拆分、可验证的搜索控制模块，而不是泛泛声称使用强化学习；把“SSHR”讲成 CNOT-oriented baseline，而不是本文方法来源；把“高维结果”讲成项集级符号验证加 bridge，而不是所有 $n = 40$ 函数都做了完整 truth-table。

表 13: 当前主张、证据与投稿边界。

主张	当前证据	边界
本文路线独立于 SSHR	状态、动作和验证都定义在 ANF/FPRM 项集合上；SSHR 只出现在 baseline 比较中。	不能声称 CNOT-only 支配 SSHR。
低 T-count 与低加权 score 是主优势	$n \leq 6$ 传统函数相对 ESOP、weighted ESOP-MILP、direct ANF 和 AND-direct ANF 有稳定优势。	资源权重仍是逻辑层人为设定。
Boolean-ring screen 是高维有效结构筛选	$n = 18, 20$ 函数级切片和 $n = 20-40$ 项集级 scale 均显示相对 single screen 的 score 改善。	高维不是全局最优性证明。
RevKit API 给出新的强外部边界	RevKit Python <code>oracle_synth</code> 在 177 个传统函数上全部可运行；在 Rz-phase lower-bound proxy 下，Resource-NMCTS 相对 RevKit score 为 6/171/0；Resource-NMCTS family 在 $\lambda_{R_z} = 1$ 时为 157/20/0，在 $\lambda_{R_z} = 1.5$ 时为 177/0/0；Ross-Selinger-style $T/R_z = 30$ proxy 下为 177/0/0； $n = 14$ timeout probe 为 1/8 返回、7/8 超时。	口径不同：RevKit 更接近 phase-rotation oracle； λ_{R_z} 和 T/R_z 都是敏感性或模型分析，不是已输出的精确 Clifford+T rotation sequence；高维 timeout 行不是资源比较行。
结构级 AI 能降低搜索成本并压缩质量 gap	Depth policy、skip guard、screen-gate、frontier policy、large frontier policy 和 cost-aware frontier policy 均有 held-out、scale 或 bridge 证据。	Large policy 是质量增强型；cost-aware policy 是快速质量折中型。
大规模结果具有语义可信度	6600/6600 项集级方法行通过 plan 和 emitted-circuit 符号验证；300/300 bridge 行通过完整 truth-table 验证。	$n = 24-40$ 仍未做完整 truth-table 枚举。
逻辑层后端相关 proxy 可解释 trade-off	schedule 泛化和 bridge 显示 frontier policy 同步降低 T-depth proxy；large policy 进一步降低 T-depth，cost-aware policy 则降低 plan time 和 lifetime area。	不是硬件 mapping、routing 或噪声模型结论。
投稿前仍需补强外部比较	当前已有 ESOP、ABC、BDD、SSHR、ROS-style LUT proxy、RevKit API baseline 和 toolchain readiness 审计。	官方 ROS、mockturtle 与 legacy RevKit/CirKit CLI 流程仍待复现。

8 讨论

本文最稳妥的贡献表述是“资源约束搜索”，不是“AI 全面战胜传统综合”。当前证据显示，ANF/FPRM 项集合提供了清晰的语义验证基础，Boolean-ring screen 提供了高维结构候选，MCTS 与神经先验提供了候选组合能力，depth policy 与 guard 控制搜索成本，depth-frontier policy 则把高预算质量前沿部分学习化。Large frontier policy 进一步说明，只要提供更充分的 frontier teacher 数据，结构级 AI 的质量 gap 可以从 0.61% 量级压到 0.10% 量级；cost-aware frontier policy 则说明同一个策略框架可以通过标签目标移动到更快的质量折中点，而不是只能追求最低 score。新增 RevKit API baseline 则提醒本文不能把“外部工具链缺失”当成唯一缺口：在

Rz-phase lower-bound 口径下, RevKit 是当前真正需要追赶的强 baseline; 但由于 171/177 行包含非 Clifford R_z , 这还不是一个可直接声称的精确 Clifford+T baseline。符号成本分析进一步说明, 该缺口并非不可追赶: 每个非 Clifford R_z 只计 1 个 score 单位时, Resource-NMCTS family 已在 157/177 行优于 RevKit, 计 1.5 个单位时覆盖全部 177 行; 传统 baseline family 在 1 个单位时只有 80/177 行优于 RevKit。近似 rotation synthesis proxy 则给出更强的边界: 只要把非 Clifford R_z 近似到 Clifford+T, RevKit lower-bound 的优势就会迅速消失; 但这个结论依赖模型常数和精度 ϵ , 不能替代真实 synthesis 输出。进一步的 $n = 14$ 隔离超时探针说明, 高维 RevKit API 结果不能靠人工观察补入论文, 而必须以 subprocess timeout 形式记录可复现边界; 当前 7/8 timeout 的证据支持把正式 RevKit paired comparison 限制在 $n \leq 6$ 。

同时, 本文必须明确四个限制。第一, SSHR 的 CNOT-oriented 优势仍存在; 在 CNOT-only 或 CNOT-depth profile 下, 本文优势会收窄。第二, RevKit API 的结果显示, 本文当前 bit-flip emitter 并不支配 phase/Rz-rotation oracle synthesis; 若论文不扩展到该口径, 就必须把主张限定清楚, 并且不能把 RevKit Rz lower bound 误写成精确 Clifford+T T-count; 即使加入 Ross-Selinger-style proxy, 也仍没有输出实际 Clifford+T rotation sequence。第三, 高维 RevKit API 探针目前只是 timeout-boundary evidence, 不是完整高维 RevKit baseline; 返回行可以讨论 gate-set 口径, timeout 行不能进入资源均值。第四, 高维神经策略的独立质量收益虽已通过 large 和 cost-aware frontier policy 变得更清楚, 但目前主要发生在结构深度选择层, 而不是任意动作级神经搜索全面优于手工结构。第五, schedule proxy 只是 emitted-circuit 层的逻辑调度代理, 本文仍没有处理硬件 mapping、连通性、routing、magic-state factory 或噪声下可靠性, 因而不能直接推出物理设备上更快或更可靠。

下一步最有价值的提升目标不是继续堆内部表格, 而是补两个外部缺口。其一, 围绕 RevKit API 暴露出的强 Rz-phase lower-bound baseline, 新增 phase/Rz-aware 的资源模型和 emitter, 并显式处理非 Clifford rotation 的精确或近似综合成本; 至少应把当前 proxy 替换为实际 gridsynth/Ross-Selinger 类 synthesis 输出的 T-count、T-depth 和 gate 序列审计, 或者把本文题目收窄为 bit-flip oracle synthesis。其二, 在 ROS-style LUT proxy 之外复现或接入官方 ROS、mockturtle 和 legacy RevKit/CirKit 流程, 使比较从 proxy/API lower bound 扩展到更标准的外部流程。同时, 当前 cost-aware 标签还应从单个时间惩罚扩展为显式多目标 teacher 或 Pareto policy, 纳入 T-depth proxy 和辅助线 lifetime area。若这些点完成, 论文主张会从“已有一条合理路线”提升到“有更强外部说服力的算法贡献”。

9 结论

本文提出 Resource-NMCTS, 将量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题, 并结合神经动作先验、MCTS、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 生成逻辑层可逆线路。实验表明, 该方法在 $n \leq 6$ 传统函数上相对 ESOP/ANF/ROS-style LUT proxy 有显著低 T-count 与低加权 score 优势, 在 $n = 18, 20$ 函数级边界上验证了 Boolean-ring screen 与 Resource tail 的互补作用, 在 $n = 20$ 到 40 项集级 scale 中展示了结构级策略的可扩展性, 并通过 $n = 21, 22, 23$ truth-table bridge 补强了高维正确性证据。Large frontier policy 将独立泛化集上相对 all-depth 的平均 score gap 压低到 0.10%, 并在 $n=23$ 完整 bridge 上相对旧 policy 进一步降低 0.48% score 和 0.45% T-depth proxy; cost-aware frontier

policy 则在保持相对 fixed depth-2 的 56/0/40 泛化胜/负/平的同时, 将相对 fixed depth-2 的 plan-time 增幅从 large policy 的 563.80% 降到 170.03%, 并在 $n=23$ bridge 上相对 large policy 节省 56.29% plan time 和 12.62% lifetime area。但 RevKit API 角度审计显示, 本文当前 X/CNOT/MCT bit-flip emitter 尚不能覆盖 Rz-phase lower-bound 口径, 而 RevKit 结果本身也不能被误读为精确 Clifford+T 成本, 因为 171/177 行包含非 Clifford R_z ; phase/Rz-cost-aware portfolio 和近似 rotation synthesis proxy 说明, 只要把非 Clifford phase cost 纳入模型, Resource-NMCTS family 即可覆盖全部 177 个传统函数。新增 $n = 14$ 隔离超时探针进一步说明, RevKit Python API 的高维比较需要被写成可复现 timeout boundary, 而不是普通 paired resource benchmark。当前稿件已经形成独立于 SSHR 的投稿主线, 但最终论文仍需围绕 RevKit/ROS/mockturtle 的外部工具链对齐、phase/Rz-aware emitter 和真正后端相关评估来提升审稿说服力。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] N. J. Ross and P. Selinger. Optimal ancilla-free Clifford+T approximation of z-rotations. *Quantum Information & Computation*, 16(11–12):901–953, 2016.
- [7] P. Selinger. Efficient Clifford+T approximation of single-qubit operators. *Logical Methods in Computer Science*, 11(2), 2015.
- [8] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [9] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [10] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.

- [11] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [12] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.